

Linear Warmup

It is used at the start of training to control the learning rate, especially in deep learning.

So instead of starting training at the target (peak) learning rate, you may start at a very small value (often zero or close to it) and increase it linearly over a fixed number of steps or epochs until it reaches the target learning rate. After the warmup ends, you typically switch to another schedule (constant, cosine decay, linear decay)

Why it is a standard

1. **Unstable early gradients:** At the start of training, weights are randomly initialized, so gradients can be large and noisy. A high learning rate applied immediately can cause the model to take huge, destabilizing steps, sometimes leading to divergence or poor final performance.
2. **Adaptive optimizers (Adam, etc.) need time to calibrate:** Optimizers like Adam maintain running estimates of gradient moments (mean and variance). Early on, these estimates are based on very few samples and are unreliable. A high learning rate combined with bad estimates can wreck training. Warmup gives these statistics time to stabilize.
3. **Batch normalization / layer normalization statistics:** Similarly, normalization layers need a few steps to get sensible running statistics; warmup avoids overreacting during that period.
4. **Common in Transformers:** Warmup became especially well known from the original Transformer paper ("Attention is All You Need"), where it's paired with an inverse square root decay afterward. It's now standard in training large language models.

The formula

If `step` is the current training step and `warmup_steps` is the number of warmup steps, and `base_lr` is your target learning rate:

```
if step < warmup_steps:
    lr = base_lr * (step / warmup_steps)
else:
    lr = base_lr # or whatever schedule follows
```

Notes

- **Typical warmup lengths:** anywhere from a few hundred steps to a few thousand, or a small percentage (e.g., 1-10%) of total training steps.
- Often combined with a decay schedule afterward: **warmup + cosine decay** and **warmup + linear decay** are very common combos.

- Warmup is more important with larger batch sizes and higher peak learning rates, since those are the settings most prone to early instability.
- It's cheap to implement and has become a near-default in training recipes for Transformers, large CNNs, and RL policies.

Note: the warmup step is often calibrated somewhere 1-10%

Pytorch.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import LinearLR, CosineAnnealingLR, SequentialLR

model = nn.Linear(10, 2)
optimizer = optim.AdamW(model.parameters(), lr=1e-3)

# Define configuration parameters
warmup_epochs = 10
total_epochs = 100

# 1. Warmup Scheduler: Linearly increase LR from (1e-3 * 0.1) to 1e-3
warmup_scheduler = LinearLR(optimizer, start_factor=0.1, end_factor=1.0,
total_iters=warmup_epochs)

# 2. Main Scheduler: Decay LR from 1e-3 down to 0 after the warmup phase
main_scheduler = CosineAnnealingLR(optimizer, T_max=total_epochs - warmup_epochs,
eta_min=0)

# 3. Combine them using SequentialLR at specified milestone
scheduler = SequentialLR(
    optimizer,
    schedulers=[warmup_scheduler, main_scheduler],
    milestones=[warmup_epochs]
)

# Training loop execution
for epoch in range(total_epochs):
    # train_epoch() code goes here
    optimizer.step()
    scheduler.step() # Call once per epoch or iteration
```

But if we want zero dependency, use the former